

Context Foundry Governance Health Check

Run this to verify the Gardener and lifecycle management are working before proceeding to enrichment phase.

Instructions for Replit

Run each query and report results. All 5 checks must pass before we proceed to relationship enrichment.

Check 1: Gardener Cycles Running

```
```sql
SELECT
 started_at,
 completed_at,
 status,
 EXTRACT(EPOCH FROM (completed_at - started_at))::int as duration_secs
FROM gardener_runs
ORDER BY started_at DESC
LIMIT 10;
```
```

**** PASS if:****

- Rows exist
- Most recent run within last 15 minutes
- status = 'success' for most runs
- duration < 180 seconds

**** FAIL if:****

- No rows (Gardener never ran)
- Last run > 15 minutes ago (Gardener stopped)
- status = 'error' consistently
- duration > 240 seconds (cycle overlapping)

Check 2: STAGING → TRUSTED Promotion

```
```sql
-- Recent promotions
SELECT
```

```

 COUNT(*) as promoted_count,
 MIN(promoted_at) as earliest,
 MAX(promoted_at) as latest
FROM entities
WHERE promoted_at > NOW() - INTERVAL '7 days'
 AND layer = 'TRUSTED';
'''

```

```

```sql
-- Promotion by day
SELECT
    DATE(promoted_at) as day,
    COUNT(*) as count
FROM entities
WHERE promoted_at > NOW() - INTERVAL '7 days'
  AND layer = 'TRUSTED'
GROUP BY DATE(promoted_at)
ORDER BY day DESC;
'''

```

**** PASS if:****

- Some promotions exist (unless no new ingestion)
- Promotion timestamps are recent

**** EXPECTED if:****

- Zero promotions when no new documents ingested recently

Check 3: Conflict Detection

```

```sql
-- Conflict summary
SELECT
 status,
 COUNT(*) as count
FROM conflicts
GROUP BY status;
'''

```

```

```sql
-- Recent conflicts
SELECT
    created_at,

```

```

        status,
        resolution_strategy,
        resolved_at
FROM conflicts
ORDER BY created_at DESC
LIMIT 10;
```

```

**\*\* PASS if:\*\***

- PENDING conflicts < 100
- Some RESOLVED conflicts exist (if any were detected)
- No stuck conflicts (PENDING for > 7 days)

**\*\* EXPECTED if:\*\***

- Zero conflicts (clean data, no contradictions detected)

---

#### ## Check 4: Demotion/Archival

```

```sql
-- Archived entities
SELECT
    COUNT(*) as archived_count,
    MIN(archived_at) as earliest,
    MAX(archived_at) as latest
FROM entities
WHERE layer = 'ARCHIVED'
    AND archived_at IS NOT NULL;
```

```

```

```sql
-- Archival by reason (if tracked)
SELECT
    DATE(archived_at) as day,
    COUNT(*) as count
FROM entities
WHERE layer = 'ARCHIVED'
    AND archived_at > NOW() - INTERVAL '30 days'
GROUP BY DATE(archived_at)
ORDER BY day DESC;
```

```

**\*\* PASS if:\*\***

- Demotion mechanism exists and has been triggered at least once
- OR no entities have decayed below threshold yet (system is new)

---

## ## Check 5: Entity Layer Distribution

```
```sql
SELECT
  layer,
  COUNT(*) as count,
  ROUND(COUNT(*) * 100.0 / SUM(COUNT(*) OVER (), 2) as percentage
FROM entities
WHERE deleted_at IS NULL
GROUP BY layer
ORDER BY
  CASE layer
    WHEN 'TRUSTED' THEN 1
    WHEN 'STAGING' THEN 2
    WHEN 'ARCHIVED' THEN 3
  END;
```
```

**\*\* PASS if:\*\***

- TRUSTED is majority (production ready)
- STAGING exists (new extractions flowing)
- Distribution makes sense for current state

**\*\*Current Expected:\*\***

- TRUSTED: ~8,500 (from prior migrations)
- STAGING: variable (depends on recent ingestion)
- ARCHIVED: small (decay hasn't had time to act)

---

## ## Check 6: Relationship Distribution

```
```sql
-- Relationship count per entity
SELECT
  CASE
    WHEN rel_count = 0 THEN '0 (isolated)'
    WHEN rel_count BETWEEN 1 AND 2 THEN '1-2 (sparse)'
    WHEN rel_count BETWEEN 3 AND 5 THEN '3-5 (adequate)'
```

```

        ELSE '6+ (rich)'
    END as bucket,
    COUNT(*) as entity_count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) as percentage
FROM (
    SELECT
        e.id,
        COALESCE(r.rel_count, 0) as rel_count
    FROM entities e
    LEFT JOIN (
        SELECT source_id as entity_id, COUNT(*) as rel_count
        FROM relationships
        WHERE layer = 'TRUSTED'
        GROUP BY source_id
        UNION ALL
        SELECT target_id as entity_id, COUNT(*) as rel_count
        FROM relationships
        WHERE layer = 'TRUSTED'
        GROUP BY target_id
    ) r ON e.id = r.entity_id
    WHERE e.layer = 'TRUSTED'
    AND e.deleted_at IS NULL
) entity_rels
GROUP BY bucket
ORDER BY
    CASE bucket
        WHEN '0 (isolated)' THEN 1
        WHEN '1-2 (sparse)' THEN 2
        WHEN '3-5 (adequate)' THEN 3
        ELSE 4
    END;
...

```

****Current Expected:****

- 0 (isolated): ~76% (~6,500 entities)
- 1-2 (sparse): ~15%
- 3-5 (adequate): ~7%
- 6+ (rich): ~2%

This distribution explains the 40% GATED rate.

Check 7: Embedding Progress

```

```sql
-- Name embedding status
SELECT
 CASE
 WHEN name_embedding IS NOT NULL THEN 'Has embedding'
 ELSE 'Missing embedding'
 END as status,
 COUNT(*) as count,
 ROUND(COUNT(*) * 100.0 / SUM(COUNT(*) OVER (), 2) as percentage
FROM entities
WHERE deleted_at IS NULL
GROUP BY (name_embedding IS NOT NULL);
```

```

****Current Expected:****

- ~9% have embeddings (batch job running)
- ~91% missing (waiting for batch job)

Summary Report Template

After running checks, provide summary:

...

GOVERNANCE HEALTH CHECK - [DATE]

Check 1 (Gardener Cycles): [PASS/FAIL] - Last run: [timestamp], Status: [status]
 Check 2 (Promotions): [PASS/EXPECTED/FAIL] - [N] promotions in 7 days
 Check 3 (Conflicts): [PASS/EXPECTED/FAIL] - [N] pending, [N] resolved
 Check 4 (Demotions): [PASS/EXPECTED/FAIL] - [N] archived entities
 Check 5 (Layer Distribution): [PASS/FAIL] - TRUSTED: [N], STAGING: [N], ARCHIVED: [N]
 Check 6 (Relationship Dist): [INFO] - Isolated: [N]%, Sparse: [N]%, Adequate: [N]%, Rich: [N]%
 Check 7 (Embedding Progress): [INFO] - [N]% complete

VERDICT: [READY FOR ENRICHMENT / NEEDS ATTENTION]

Issues Found:

- [List any issues]

Recommendations:

- [List any recommendations]

...

Next Steps After Health Check

****If PASS:****

1. Proceed to Week 2 Response Modes
2. Let batch embeddings complete in background
3. Plan validation UI before enrichment

****If FAIL:****

1. Diagnose failed component
2. Fix before proceeding
3. Re-run health check